

BNF Parser Assignment Report (Assignment 2)

FIT 2102

Jonethe Tan Yang, 33647739

Design & Architecture:

- **High Level Approach**

The implementation of the code basically follows a stage pipeline where:

1. Parse BNF
2. Validate
3. Generate the Haskell code

This is done to ensure separation of concerns and to enable independent testing along with modification during the coding process without the need to worry about other functions being affected.

- **Data Structure Design**

The ADT hierarchy is done to mirror the BNF structure that is:

Grammar > Rule > Production > Alternative > Component

So, from the code the component type uses a design consisting of three orthogonal aspects: that is a Boolean Token, prefix Flag and lastly a base component Type (Nonterminal (NTs) / Terminals (Ts)) , and a modifier (Optional/ zero or more / one or more). This design is done to avoid the combinatorial confusion of data constructors (Which can go up to 12+ constructors mind I add) while maintaining type safety along with clarity.

- **Key Architectural Choices**

- **Separation of concerns in components** (Aka a lot of helpers):
Token prefix handling is separated from base parsing; this is mainly done to allow the clean composition along with a simpler and easier to debug code generation logic
- **Iterative Validation** (Or flowing if you like):

The validator iteratively removes the problematic rules (Dupes, undefined refs, left recursion etc.) until it reaches a fixed point thus ensuring transitive dependencies are handled properly

- **Smart Type Generation** (I mean it's not stupid):
Single alternative rules generate (newType) declaration while multi-alts rule generate data types, thus producing idiomatic Haskell code.

Parsing with Combinators

- **Why is Parser combination used here:**

Parser combination is ideal for BNF parsing, this is due to them mirroring grammar structure directly. The code (**productionParser = Alts <\$> sepPipe parseAlternative**) is done so as it reads similarly to BNF specs itself. Composability allows the building of complex parsers from an array of simple primitives (Like Lego) without the need for a manual state management or backtracking logic

Functor/Applicative/Monad

- **Functor (<\$>):** Maps in the code are constructed over the parsed values. (**Grammer <\$> many rParse**) is there to transform a list of parsed rules into the Grammer ADT. This is done so to eliminate the need for explicit lambda functions
- **Applicative (<*>):** Sequence parsers while accumulating results. (**rBuilder <\$> lhsRuleParse <*> rhsRuleParse**) parses the left-handed side, then the right-handed side, applying both results to the rule builder. This is done so as its cleaner than the monadic bind chains when we don't need to inspect the intermediate results.
- **Sequential Composition (*>, <*>):** Parsers and discards the unwanted token, (**is '<' *> p <*> is '>'**) parses the angle bracket but keeps only the content in the middle. This is done so I don't have to use bind and explicitly ignore the values itself
- **Alternative (<|>):** Tries parsers in a sequence until one of them succeeds. (**nTParse <|> tParse <|> mcoParse**). Attempts each component type. These implements backtrack elegantly without the need for explicit conditionals (better do it elegantly then needing to explain tons of spaghetti code)

Key Parser Design Choice

- **Whitespace handling:** Explicit control via the **(inSpacer0)** and **(inSpacer1)** for required versus optional spaces. This precision prevents ambiguous passes.
- **Separator Abstraction:** The generic **SepBy1** function is then parameterized over separator type, this is done so to enable to reuse for commas, pipes alongside whitespace. This demonstrates higher order function
- **Newline Prevention:** The **neinNewLine** parser ensures components stay on the same line, critical for maintaining proper grammar structure without allowing the arbitrary line breaks mid-prods

Functional Programming

- **Modular Function Composition:**
Small, single purpose functions are composed to handle complexity elegantly. For example **(validRuleFltr = filterIter. dupliRem)** shows point free composition where each function does one thing at a time thus makes the validation pipeline obvious.
- **Declarative Over Imperative**
Instead of loops and mutation, higher order function is used **(getUsedNTs rules = concatMap usedNTsInRule rules)** declares what we need (all used up nonTerms) rather than how to iterate, so list comprehension like **(name | Rule name _ _)** extract data declaratively, improving readability.
- **Why small Functions?** (it gets the job done as the big boys)
 - **Testability:** Each function has clear inputs/outputs, easy to test in isolation. **Capitalizor** or **CountOccurrences** can be verified independently
 - **Reusability:** **SepBy1** works for any separator; **Capitalizor** is used in type generation, constructing naming and parser generation
 - **Reasoning:** Pure functions with no side effects make control flow explicit. **BuildRuleGraph** clearly constructs a graph; its output is then used directly by **reachChckr**

Haskell Language Features

- **Custom Types and Pattern Matching:** The ADT design leverages Haskell's algebraic data types. Pattern matching on **Component _ (CNonTerminal name _)**

extracts nonterminal cleanly without nested conditionals. The deriving show clause automatically generates debugging output

- **Higher Order Functions:** Function like (**concatMap**, **filter**, **nubBy**) abstract iteration patterns. **nubBy nameRuleComp** removes duplicates using a custom comparison function, showing how behavior can be parametrized.
- **Function Composition:** The . Operator chains transformation: **validRuleFltr = filterlter. DupliRem**. This is to read left to right conceptually (Remove duplicates, then filter) while being point free
- **Type Classes:** The parser type implements **Functor** and **Applicative**, enabling the combinator style. Without these abstractions, we'd need to explicitly pattern matching **Result** and **Error** constructors everywhere.

EXTENSIONS (Its quite boring actually)

- **ESCAPE CHAR SUPPORT**
- **What was implemented:** Full escape sequence handling in terminal strings including `\n`, `\t`, `\r`, `\\`, `\"`, `\'`.
- **How it works:** The `charInQuote` parser tries `escChar` before `regChar`. `escChar` parses a backslash followed by `escCodeToChar`, which pattern matches on escape codes and returns the corresponding character.
- **Why it's interesting:** This required careful parser design to handle backslash as both an escape introducer and a literal character (when escaped). The modular design (`escChar` separate from `regChar`) makes the logic clear and extensible for additional escape sequences.
- **Complexity:** Handling nested parsing (backslash then escape code) while maintaining the correct position in the input stream demonstrates the power of parser combinators for context-dependent parsing

